



Московский
транспорт

Департамент транспорта и развития
дорожно-транспортной инфраструктуры

Контроль препятствий по данным 3D-лидара

HW · Rail Inspector

Лидеры цифровой трансформации 2026



О команде

Капитан: Генрих Вирт, ML-инженер

Участников: 2

ML-разработка и анализ

- *пространственных данных*
- *Москва*

Rail Inspector

Краткое описание решения:

Строим путь по рельсам и проверяем, какие точки пересекают габарит поезда.

Уникальность решения:

Геометрический метод без обучения на классах препятствий. Контактный рельс служит опорой для дальнего пути.

КОМАНДА



**Вирт Генрих
Вальтерович**

- ML-инженер, капитан
- @HeinrichWirth
- Центральный университет
- Геограком: тимлид,
- инженер-исследователь



**Потапов Михаил
Алексеевич**

- Пространственные данные
- @Sam_Wolfhound
- МГУ
- Учитель в школе

Краткая история команды:

01

Мы знакомы и хорошо работаем вместе.

Это наш первый совместный хакатон.

Почему вы выбрали именно эту задачу из предложенных на хакатоне?

02

Задача соединяет ML-инженерию и работу с пространственными данными.

С какими основными сложностями или вызовами вы столкнулись и как их преодолели?

03

Перекрытия рельсов, неоднозначная разметка и редкие дальние точки. Мы разделили поиск рельсов, проверку продолжения и подтверждение объектов. Каждый этап проверяли на сохранённых наблюдениях.

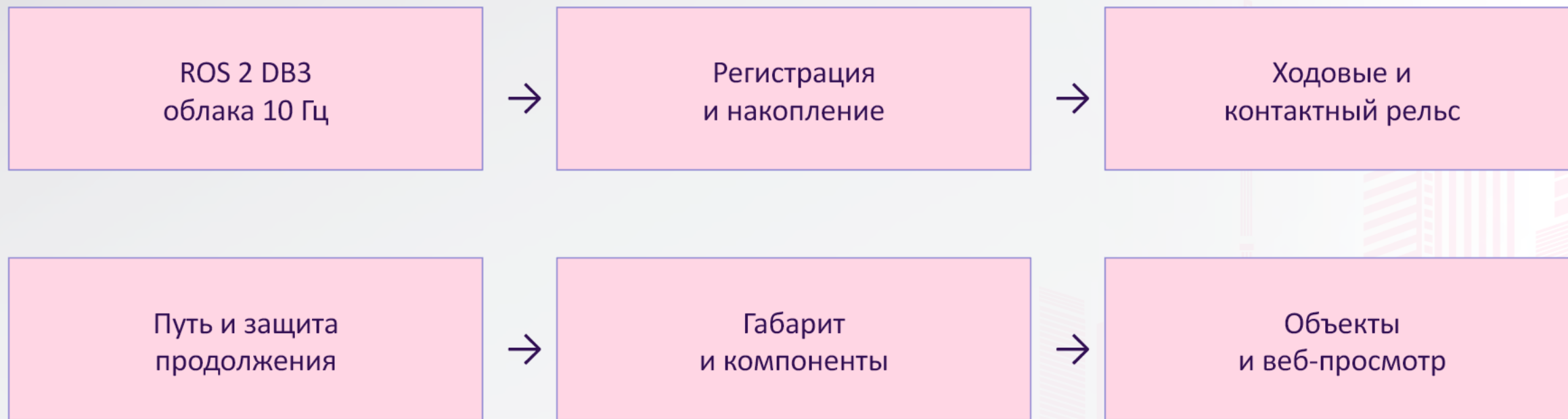
Техническая суть решения

- Поиск двух ходовых рельсов на 0–8 м.
- Контактный рельс задаёт дальнюю геометрию пути. При его отказе корректируем прошлый путь по ближним рельсам.
- Проверяем точки в габарите 2,1 × 3 м. Объекту нужны наблюдения минимум на двух кадрах.

Маркетинговая суть решения

- Первое применение: разбор проездов и проверка обнаружений инженерами метро.
- Следующий этап: подключение к ROS 2 Humble и испытания на контрольном стенде.
- Развитие: предварительная карта пути и локализация поезда для сокращения повторного поиска рельсов.

Архитектура расчёта



Полный расчёт 3 Гц. Промежуточные облака пополняют историю.

Для направления используем позу T+1. Будущие облака в поиск не входят.

Геометрия вместо классов объектов



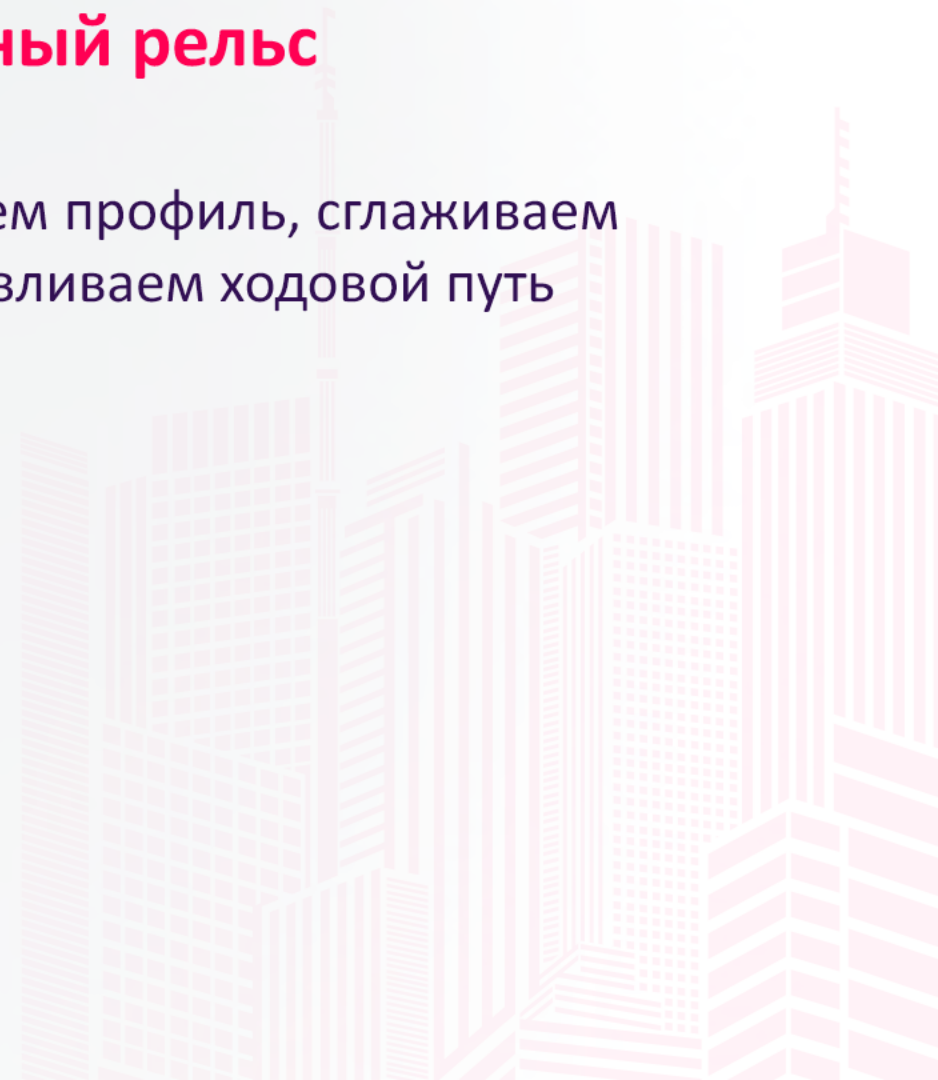
0–8 м

Независимый поиск головок
двух ходовых рельсов

При отказе контактного поиска используем прошлый путь
с коррекцией по текущим ходовым. Защита отсекает
продолжение без поддержки реальными точками.

Контактный рельс

Продолжаем профиль, сглаживаем
и восстанавливаем ходовой путь



Решение о препятствии



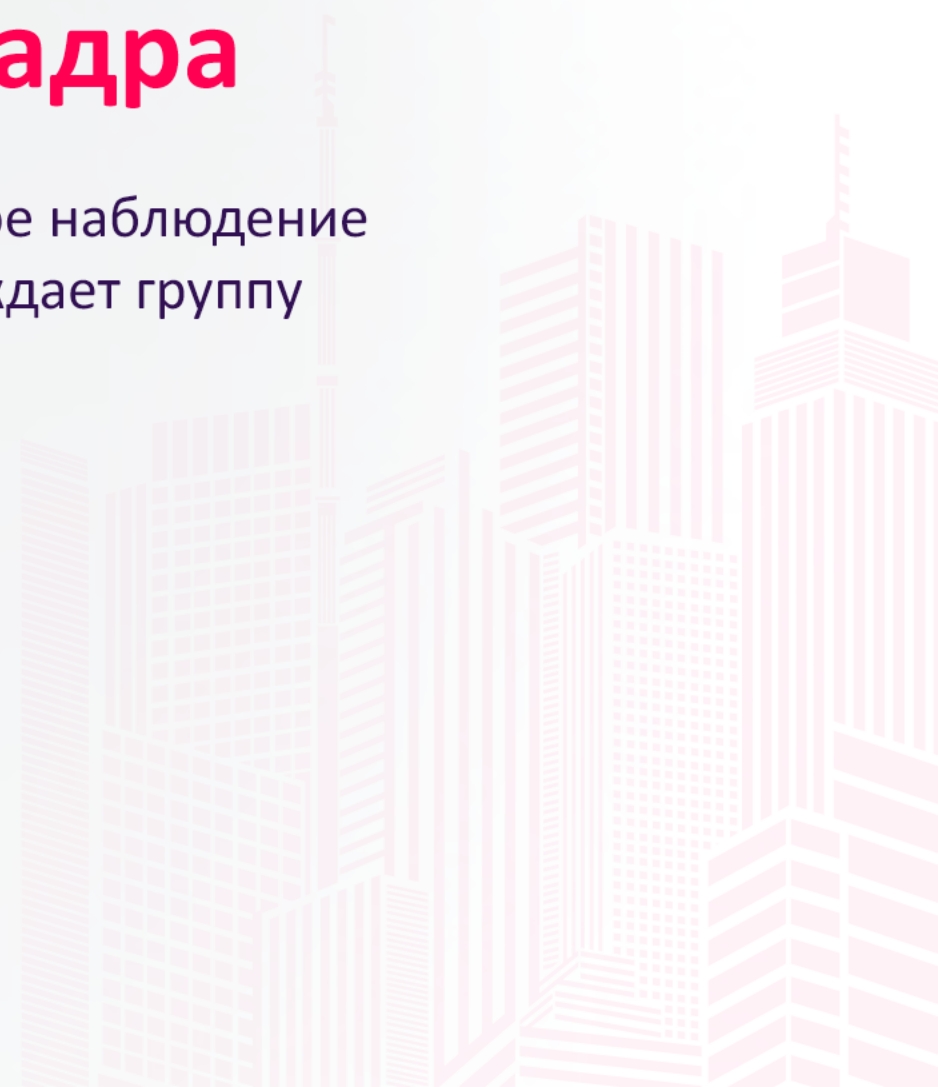
2,1 × 3 м

Габарит движется вдоль
предсказанных рельсов

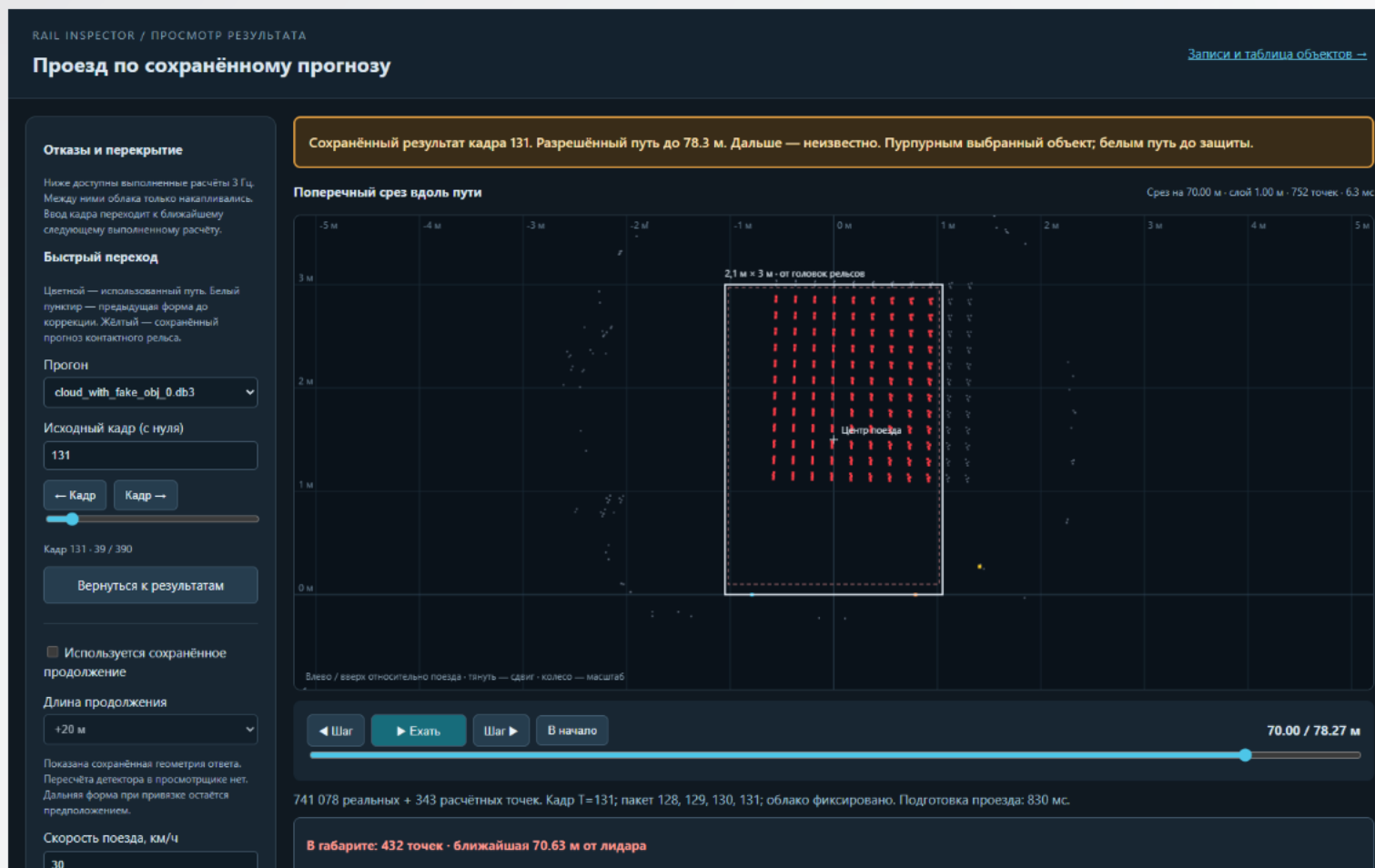
≥ 2 кадра

Повторное наблюдение
подтверждает группу

Точки внутри габарита образуют компоненты.
Ближние наблюдения объединяем в радиусе 10 м.
После 70 м уменьшаем вклад отдельных точек.



Обнаружение и дальность



Кадр 131: первое наблюдение группы на 70,63 м от лидара

Проверка результата в браузере

Объект	Кадры	Ближайшее	Ш × В	Площадь	
#1	131–214	16.17 м	1.90 × 1.61 м	3.044 м ²	Посмотреть
#3	347–361	15.30 м	0.38 × 0.33 м	0.127 м ²	Посмотреть
#4	394–424	5.19 м	0.37 × 0.21 м	0.076 м ²	Посмотреть
#5	467–477	7.15 м	0.09 × 0.34 м	0.032 м ²	Посмотреть
#6	494–524	17.89 м	0.11 × 0.29 м	0.033 м ²	Посмотреть
#8	597–607	55.75 м	0.07 × 1.94 м	0.134 м ²	Посмотреть
#10	640–684	3.94 м	2.04 × 0.14 м	0.294 м ²	Посмотреть
#11	697–744	3.22 м	1.82 × 0.20 м	0.373 м ²	Посмотреть
#12	788–804	5.24 м	0.06 × 0.27 м	0.016 м ²	Посмотреть

9 подтверждённых групп. У каждой есть кадры, дальность и размер.

Скорость полного прогона

262 мс

медиана расчёта

1510 входных кадров
390 завершённых расчётов

310 мс

95-й процентиль

CPU, без GPU
Планирование 3 раза в секунду

Время включает подготовку и геометрию, без записи диагностики.

Проверка воспроизводимости



317 / 317

Совпадение геометрии
на одинаковых входах

356 / 356

Совпадение индексов
точек внутри габарита

Docker собирает C++ из исходников.
Проверены загрузка DB3, расчёт и веб-просмотр.
Контрольные данные заказчика остаются отдельным испытанием.

Развитие: карта предварительного проезда

Сейчас

Строим путь из текущих
и прошлых наблюдений.
Перекрытия ограничивают
дальность принятого пути.

С предварительной картой

Локализуем поезд на пути.
Сравниваем новые точки
с известным габаритом
и геометрией тоннеля.

Гипотеза: меньше повторного поиска рельсов.
Нужно проверить локализацию, изменения карты и скорость на стенде.

Запуск и демонстрация



github.com/HeinrichWirth/deptranshack

```
docker compose up --build -d
```

localhost:8870

Загрузить DB3 и начать расчёт

Открыть объект и проехать по прогнозу

Контакт команды: @HeinrichWirth

